

GitHub Learning Material

1. Git Introduction

Git is a distributed version control system that helps developers track changes in their code, collaborate on projects, and manage versions of their software. GitHub is a platform built on Git that provides a collaborative environment for managing repositories in the cloud.

Key Features of Git:

- Tracks code changes and history.
- Supports branching and merging for collaborative work.
- Provides tools for resolving code conflicts.

Why GitHub?

- Centralized repository for easy access and sharing.
 - Collaboration tools like pull requests and issue tracking.
 - Integration with CI/CD pipelines and project management tools.
-

2. Repository Creation Using CLI

A repository (repo) is a storage space for project files, including code and documentation.

Steps to Create a Repository Using CLI:

1. **Initialize a Local Repository:**
2. `git init`

This command sets up a Git repository in your local project directory.

3. **Create a Remote Repository on GitHub:**

- Log in to GitHub.
- Click on **New Repository** and provide a name.

4. Link Local Repo to Remote Repo:

5. `git remote add origin <repository_url>`

6. Push Initial Code:

7. `git add .`

8. `git commit -m "Initial commit"`

9. `git push -u origin main`

3. Git Clone and Fork Tasks

Clone: Copies an existing repository to your local machine.

`git clone <repository_url>`

Fork: Creates a personal copy of another user's repository on GitHub for independent modifications.

Common Use Case:

- Clone a repository to work locally.
 - Fork a repository to contribute to open-source projects or teams.
-

4. Branching in Git

A branch is a separate line of development.

Why Use Branches?

- To work on new features or fixes without affecting the main codebase.
- To collaborate with team members.

Commands for Branching:

- **Create a New Branch:**
 - `git branch <branch_name>`
- **Switch to a Branch:**
 - `git checkout <branch_name>`
- **Create and Switch Simultaneously:**
 - `git checkout -b <branch_name>`

5. Git Merge and Tagging

Merge: Combines changes from one branch into another.

`git merge <branch_name>`

Tagging: Marks specific points in history for releases or milestones.

- Create a Tag:
- `git tag -a <tag_name> -m "Tag message"`
- Push Tags to Remote:
- `git push origin <tag_name>`

Use Case:

- Merge a feature branch into the main branch.
 - Tag a commit for a software release.
-

6. Git Pull and Fetch Commands

Pull: Fetches changes from a remote repository and merges them into your local branch.

`git pull`

Fetch: Retrieves changes from the remote repository but does not merge them.

`git fetch`

Difference:

- `git pull = git fetch + git merge.`
 - Use fetch when you want to review changes before merging.
-

7. Git Stash and Git Cherry-Pick

Stash: Temporarily saves uncommitted changes.

- Save Changes:
- `git stash`

- Apply Stashed Changes:
- `git stash apply`

Cherry-Pick: Applies specific commits from one branch to another.

`git cherry-pick <commit_hash>`

Use Case:

- Use stash when switching tasks without committing incomplete work.
 - Use cherry-pick to transfer a critical bug fix to another branch.
-

8. Pull Requests

Pull Requests:

- A way to propose changes to a repository.
- Enables code review and discussion before merging.

Steps to Create a Pull Request:

1. Push your branch to the remote repository.
2. Go to the repository on GitHub.
3. Click **New Pull Request**.
4. Provide a description and submit the request.

Common Use Case:

- Propose new features or fixes.
 - Collaborate with team members on code reviews.
-

9. Additional Best Practices

1. **Write Clear Commit Messages:**

- Example: Added user authentication feature.

2. **Use .gitignore:**

- Exclude unnecessary files from being tracked.
- Example: Logs, environment files.

3. Collaborate Effectively:

- Use branches for different features.
- Regularly pull changes to stay updated.

4. Code Reviews:

- Use pull requests for collaborative feedback.

10. Hands-On Tasks

Task 1: Create a Repository and Push Code

- Initialize a Git repo locally.
- Create a remote repo on GitHub and push your code.

Task 2: Practice Branching and Merging

- Create a branch for a new feature.
- Make changes, commit, and merge the branch back to main.

Task 3: Experiment with Stash

- Make uncommitted changes and stash them.
 - Retrieve the stashed changes.
-